



PROYECTO ENTRADA/SALIDA
MEMORIA
ARQUITECTURA DE COMPUTADORES



AUTORES

Fernando Luna Marcuello, 190394

Borja Pérez-Villacastín Palacín, 190395

ÍNDICE

1. Diagrama de flujo o pseudocódigo.....	3
1.1. SCAN.....	3
1.2. PRINT.....	4
1.3. RTI.....	5
2. Subrutinas en ensamblador.....	6
2.1. INIT.....	6
2.2. SCAN.....	7
2.3. PRINT.....	8
2.4. RTI.....	9
3. Casos de prueba.....	11
3.1. Casos de prueba para INIT.....	11
3.2. Casos de prueba para SCAN.....	11
3.3. Casos de prueba para PRINT.....	14
3.4. Casos de prueba para RTI.....	16
4. Conclusiones.....	18

1. Diagrama de flujo o pseudocódigo

Comenzamos el proyecto planteando las subrutinas propuestas como si fueran funciones a implementar en algún lenguaje que dominamos, como podía ser C. Por lo tanto, antes de comenzar la programación en ensamblador, escribimos las posibles funciones en C de las subrutinas SCAN, PRINT y RTI. Consideramos que no era necesario escribir un pseudocódigo de la subrutina INIT, ya que en la documentación del proyecto estaba muy claro los pasos a seguir para su implementación directamente en el ensamblador.

1.1. SCAN

```
int scan(int buffer, int descriptor, int tamaño){  
    if((descriptor<0)||((descriptor>1)||((tamaño<0)) return -1;  
    if(tamaño==0) return 0;  
    int D0, restantes;  
    restantes=tamaño;  
    while(restantes!=0){  
        D0=leecar(descriptor);  
        if(D0==-1) break;  
        meter(D0, buffer);    //D0-->buffer  
        restantes--;  
        buffer++;  
    }  
    D0=tamaño-restantes;  
    return D0;  
}
```

Primero de todo, comenzamos SCAN comprobando si alguno de los parámetros que se han pasado es erróneo, en tal caso la función devolverá un -1 (0xffffffff) por D0. También se comprobará si el tamaño recibido es igual a cero para terminar directamente la rutina.

A continuación, comenzará el bucle de lectura de caracteres. En cada iteración de este se leerá un caracter del buffer de recepción de la línea correspondiente, para posteriormente, meter ese caracter leído en el buffer dado, decrementar el número de caracteres que faltan por leer y mover el puntero del buffer a la siguiente posición. Este bucle terminará cuando el número de caracteres que quedan por leer sea cero, o cuando LEECAR no consiga leer más caracteres.

Finalmente se calculará el número de caracteres leídos en D0 y finalizará la rutina.

1.2. PRINT

```
int print(int buffer, int descriptor, int tamaño){
    if((descriptor<0)||((descriptor>1)||((tamaño<0)) return -1;
    if(tamaño==0) return 0;
    int D0, D1, restantes;
    restantes=tamaño;
    descriptor=descriptor+2;
    while(restantes!=0){
        D1=cargar(buffer);          //D1←-(buffer)
        if(D1==0) break;
        D0=esccar(descriptor,D1);
        if(D0==-1) break;
        restantes--;
        buffer++;
    }
    D0=tamaño-restantes;
    if(D0!=0){
        deshabilitoINTS();
        if(descriptor==2) setMascara(TxRDYA, 1);
        else setMascara(TxRDYB, 1);
        habilitoINTS();
    }
    return D0;
}
```

Igual que en SCAN, se comienza comprobando si alguno de los parámetros que se pasan a PRINT es erróneo, de ser así se devolverá -1 (0xffffffff) en D0.

Posteriormente, se sumará dos al descriptor que recibe PRINT para que este apunte al correspondiente buffer de transmisión de cada línea y dará comienzo el bucle de escritura. En cada iteración de este bucle se cargará un carácter en D1 y el descriptor en D0 para llamar a ESCCAR, aunque antes de eso, se comprueba que el carácter que se carga en D1 no sea nulo. Después de la llamada a ESCCAR, se comprobará que haya escrito el caracter y decrementará el número de caracteres restantes por escribir, además de mover el puntero del buffer del que se leen caracteres. Este bucle puede finalizar de tres formas; que el número de caracteres restantes sea llegue a cero, que ESCCAR no consiga escribir más caracteres, que el buffer del que extraemos caracteres no tenga ningún caracter más.

Al terminar el bucle, se calcula en D0 el número de caracteres escritos y, en caso de no haber escrito ninguno, terminará la rutina. En el caso contrario, antes de finalizar, serán activadas las interrupciones de la correspondiente línea modificando la Máscara de Interrupción.

1.3. RTI

```
void rti(){
    while(TxRDYA || RxRDYA || TxRDYB || RxRDYB){
        if(TxRDYA==1){
            D0=leecar(2);
            if(D0!=-1) meter(D0,TBA);
            else setMascara(TxRDYA, 0);
        }
        if(RxRDYA==1){
            D1=cargar(RBA);
            D0=esccar(0, D1);
        }
        if(TxRDYB==1){
            D0=leecar(3);
            if(D0!=-1) meter(D0,TBB);
            else setMascara(TxRDYB, 0);
        }
        if(RxRDYB==1){
            D1=cargar(RBB);
            D0=esccar(1, D1);
        }
    }
}
```

RTI tratará las interrupciones mediante un bucle, el cual, terminará únicamente cuando todos los bits de interrupción del Registro de Estado de Interrupción enmascarado con la Máscara de Interrupción estén a 0.

En cada iteración de este bucle se comprueba qué bits de interrupción están habilitados y los se tratan de manera específica. En el caso de interrupciones de transmisión, leerá un carácter del buffer de transmisión de la línea correspondiente para posteriormente meterlo en su Registro de Transmisión. Si no se consigue leer más caracteres de la línea, cambiará la máscara de interrupción para desactivar el bit TxRDY de esa línea. Por otro lado, la recepción sería más simple puesto que únicamente se carga el carácter del Registro de Recepción y lo escribe con ESCCAR en el buffer de recepción, siempre en su correspondiente línea.

2. Subrutinas en ensamblador

Para la implementación de las subrutinas en ensamblador hemos utilizado las herramientas Notepad++ y Visual Studio Code, esta última con la extensión M68k-Assembly para diferenciar mediante colores entre etiquetas, instrucciones y comentarios.

Para la compilación del código, primero pasamos el código de Windows a la máquina virtual triqui utilizando la herramienta WinSCP. Luego, compilamos utilizando el escritorio virtual de la UPM, iniciando una sesión remota a triqui con SSH y habilitando la conexión X11, para ello usamos la herramienta PuTTY. De esta manera, podemos compilar y ejecutar nuestro código en el emulador bsvc que está instalado en triqui.

A continuación, escribimos el código en ensamblador de las subrutinas que hemos implementado en el proyecto: INIT, SCAN, PRINT y RTI.

2.1. INIT

```
INIT    MOVE.B    #%00010000,CRA    *Reiniciamos puntero a Registro de Modo 1 Linea A
        MOVE.B    #%00010000,CRB    *Reiniciamos puntero a Registro de Modo 1 Linea B
        MOVE.B    #%00000011,MR1A    *Establecemos 8 bits por caracter Linea A
        MOVE.B    #%00000011,MR1B    *Establecemos 8 bits por caracter Linea A
        MOVE.B    #%00000000,MR2A    *Modo de operacion Normal Linea A (No Eco)
        MOVE.B    #%00000000,MR2B    *Modo de operacion Normal Linea B (No Eco)
        MOVE.B    #%11001100,CSRA    *Establecemos velocidades Linea A (38400bps)
        MOVE.B    #%11001100,CSRB    *Establecemos velocidades Linea B (38400bps)
        MOVE.B    #%00000000,ACR    *Establecemos Conjunto 0 para Registros de Seleccion de Reloj A y B
        MOVE.B    #%00010101,CRA    *Transmision y recepcion activados Linea A
        MOVE.B    #%00010101,CRB    *Transmision y recepcion activados Linea B
        MOVE.B    #%00100010,CIMR    *Habilitamos Recepcion Linea A y B en la copia de Mascara de Interrupcion
        MOVE.B    CIMR,IMR          *Pasamos la copia a la propia Mascara de Interrupcion
        MOVE.B    #$40,IVR          *Establecemos el Registro de Vector de Interrupcion como 40 en hexadecimal
        MOVE.L    #RTI,$100         *Ponemos la direccion de RTI en la tabla de vectores de excepcion
        BSR      INI_BUFS          *Inicializamos bufferes
        RTS                        *Retorno de subrutina
```

2.2. SCAN

```
SCAN    LINK A6,#0                *Creamos el marco de pila
        MOVEM.L A0/D1-D2,-(A7)    *Guardamos en pila el contenido de los registros que vamos a utilizar
        CLR.L D0                  *Limpiamos el contenido de los registros
        MOVE.W 14(A6),D1          *Carga TamaNo en D1
        MOVE.W 12(A6),D2          *Carga Descriptor en D2
        MOVE.L 8(A6),A0           *Carga dir del Buffer en A0
        CMP.W #1,D2               *Comparamos para ver si descriptor incorrecto
        BGT ERRSC                 *Si Descriptor>1 saltamos a Error
        CMP.W #0,D2               *Comparamos para ver si descriptor incorrecto
        BLT ERRSC                 *Si Descriptor<0 saltamos a Error
        CMP.W #0,D1               *Comparamos para ver si Tamano incorrecto
        BLT ERRSC                 *Si TamaNo<0 saltamos a Error
        BEQ FINSC                 *Si TamaNo==0 vamos al final de la subrutina
BUCSC   CMP.W #0,D1               *Comparacion para fin de bucle
        BEQ FBUCSC                *Salimos del bucle si el TamaNo==0
        EOR.L D0,D0               *Limpiamos D0
        MOVE.W D2,D0              *Cargamos descriptor en D0
        BSR LEECAR                 *Llamamos a LEECAR para que nos devuelva en D0 el caracter
        CMP.L #$FFFFFFFF,D0        *Vemos si LEECAR ha leído algun caracter
        BEQ FBUCSC                *Si no ha leído caracter salimos del bucle
        SUB.W #1,D1               *Decrementamos el Tamano
        MOVE.B D0,(A0)+            *Caracter-->(Buffer), Buffer+1
        BRA BUCSC                 *Saltamos comienzo bucle
FBUCSC  EOR.L D0,D0               *Limpiamos D0. Puede que LEECAR haya devuelto 0xffffffff
        MOVE.W 14(A6),D0           *Carga TamaNo en D0
        SUB.W D1,D0                *TamanoOriginal - TamanoRestante = NumCarLeidos
        BRA FINSC                 *Saltamos al final de la subrutina
ERRSC   MOVE.L #$FFFFFFFF,D0        *Si ha habido algun error en los parametros de entrada cargamos 0xffffffff en
                                     D0
FINSC   MOVEM.L (A7)+,A0/D1-D2     *Restauramos los valores de los registros previos a la llamada de la subrutina
        UNLK A6                    *Destruimos el marco de pila
        RTS                        *Retorno de subrutina
```

2.3. PRINT

```
PRINT  LINK A6,#0           *Creamos el marco de pila
      MOVEM.L A0/D1-D3,-(A7) *Guardamos en pila el contenido de los registros que vamos a utilizar
      CLR.L D0              *Limpiamos el contenido de los registros
      MOVE.W 14(A6),D3      *Carga TamaNo en D3
      MOVE.W 12(A6),D2      *Carga Descriptor en D2
      MOVE.L 8(A6),A0       *Carga dir del Buffer en A0
      CMP.W #1,D2          *Comparamos para ver si descriptor incorrecto
      BGT ERRPR            *Si Descriptor>1 saltamos a Error
      CMP.W #0,D2          *Comparamos para ver si descriptor incorrecto
      BLT ERRSC            *Si Descriptor<0 saltamos a Error
      ADD.W #2,D2          *Anadimos 2 al descriptor para que escriba en buffer transmision de la
                           correspondiente linea

      CMP.W #0,D3          *Comparamos para ver si Tamano incorrecto o 0
      BLT ERRPR            *Si TamaNo<0 saltamos a Error
      BEQ FINPR            *Si TamaNo==0 vamos al final de la subrutina
BPR    CMP.W #0,D3          *Comparacion para fin de bucle
      BEQ FBPR             *Salimos del bucle si el TamaNo==0
      MOVE.W D2,D0         *Cargamos Descriptor en D0
      MOVE.B (A0),D1       *(Buffer)-->D1 , llevamos el caracter a D1 para que lo utilice ESCCAR
      CMP.B #0,D1          *Comprobamos si el caracter es nulo
      BEQ FBPR             *Si es nulo salimos del bucle
      BSR ESCCAR           *Llamamos a ESCCAR
      CMP.L #$FFFFFFFF,D0  *Vemos si ESCCAR ha escrito algun caracter
      BEQ FBPR             *Si no ha escrito caracter salimos del bucle
      SUB.W #1,D3          *Decrementamos el TamaNo
      ADD.W #1,A0          *Movemos puntero Buffer
      BRA BPR              *Saltamos comienzo bucle
FBPR   EOR.L D0,D0         *Limpiamos D0. Puede que ESCCAR haya devuelto 0xffffffff
      MOVE.W 14(A6),D0     *Carga TamaNo en D1
      SUB.W D3,D0          *Tamaño Original - Tamaño Restante = Num CarEscritos
      CMP.W #0,D0         *Comparamos si hemos caracteres
      BEQ FINPR            *Si no hemos escrito caracteres salimos
      CMP.W #3,D2          *Hemos escrito caracteres. Vemos en que linea debemos activar la transmision
      BEQ TRNSB            *Saltamos para activar la transmision de B o de lo contrario seguimos para
```

		activar A
TRNSA	BSET #0,CIMR	*Activamos bit de transmision de A en la copia de la Mascara de Interrupcion
	BRA TINT	*Saltamos para no activar B tambien
TRNSB	BSET #4,CIMR	*Activamos bit de transmision de B en la copia de la Mascara de Interrupcion
TINT	MOVE.W SR,D1	*Guardamos el Registro de Estado actual
	MOVE.W #\$2700,SR	*Deshabilitamos las interrupciones
	MOVE.B CIMR,IMR	*Pasamos la copia de la macara que hemos editado a la Mascara de Interrupcion
	MOVE.W D1,SR	*Restauramos el Registro de Estado
	BRA FINPR	*Saltamos al final de la subrutina
ERRPR	MOVE.L #\$FFFFFFFF,D0	*Si ha habido algun error en los parametros de entrada cargamos 0xffffffff en D0
FINPR	MOVEM.L (A7)+,A0/D1-D3	*Restauramos los valores de los registros previos a la llamada de la subrutina
	UNLK A6	*Destruimos el marco de pila
	RTS	*Retorno de subrutina

2.4. RTI

RTI	MOVE.L D0,-(A7)	*Guardamos registros D0 y D1 en pila para poder alterar su contenido
	MOVE.L D1,-(A7)	
BUCRTI	MOVE.B ISR,D0	*Cargamos ISR en D0
	AND.B CIMR,D0	*Lo unimos con la Mascara de Interrupcion para ver que interrupciones tenemos que tratar
TRDYA	BTST #0,D0	*Si bit de TxREADY A a 0 salta a comprobar bit RxREADY A0
	BEQ RRDYA	*Si no salta, comenzamos a tratar la interrupcion de transmision por la linea A
	MOVE.L #2,D0	*Cargamos un dos como descriptor en D0 (Buffer de transmision de A)
	BSR LEECAR	*LLamamos a LEECAR
	CMP.L #\$FFFFFFFF,D0	*Vemos si ha leído caracter
	BNE NOULTA	*Si ha leído caracter salta
	BCLR #0,CIMR	*Desactivamos bit de transmision de A en la copia de la Mascara de Interrupcion
	MOVE.B CIMR,IMR	*Pasamos la copia de la macara que hemos editado a la Mascara de Interrupcion
	BRA BUCRTI	*Saltamos al bucle de RTI para comprobar si hay interrupciones activas
NOULTA	MOVE.B D0,TBA	*Si ha leído caracter lo movemos al Registro de Transmision de A
	BRA BUCRTI	*Saltamos al bucle de RTI para comprobar si hay interrupciones activas
RRDYA	BTST #1,D0	*Si bit de RxREADY A a 0 salta a comprobar bit TxREADY B
	BEQ TRDYB	*Si no salta, comenzamos a tratar la interrupcion de recepcion por la linea A
	MOVE.B RBA,D1	*Cargamos en D1 un caracter del Registro de Recepcion de A

	MOVE.L #0,D0	*Cargamos en D0 un 0 como descriptor
	BSR ESCCAR	*LLamamos a ESCCAR
	BRA BUCRTI	*Saltamos al bucle de RTI para comprobar si hay interrupciones activas
TRDYB	BTST #4,D0	*Si bit de TxREADY B a 0 salta a comprobar bit RxREADY B
	BEQ RRDYB	*Si no salta, comenzamos a tratar la interrupcion de transmision por la linea B
	MOVE.L #3,D0	*Cargamos un dos como descriptor en D0 (Buffer de transmision de B)
	BSR LEECAR	*LLamamos a LEECAR
	CMP.L #\$FFFFFFF,D0	*Vemos si ha leído caracter
	BNE NOULTB	*Si ha leído caracter salta
	BCLR #4,CIMR	*Desactivamos bit de transmision de B en la copia de la Mascara de Interrupcion
	MOVE.B CIMR,IMR	*Pasamos la copia de la macara que hemos editado a la Mascara de Interrupcion
	BRA BUCRTI	*Saltamos al bucle de RTI para comprobar si hay interrupciones activas
NOULTB	MOVE.B D0,TBB	*Si ha leído caracter lo movemos al Registro de Transmision de B
	BRA BUCRTI	*Saltamos al bucle de RTI para comprobar si hay interrupciones activas
RRDYB	BTST #5,D0	*Si bit de RxREADY B a 0 salta al final de RTI
	BEQ FINRTI	*Si no salta, comenzamos a tratar la interrupcion de recepcion por la linea B
	MOVE.B RBB,D1	*Cargamos en D1 un caracter del Registro de Recepcion de B
	MOVE.L #1,D0	*Cargamos en D0 un 1 como descriptor
	BSR ESCCAR	*LLamamos a ESCCAR
	BRA BUCRTI	*Saltamos al bucle de RTI para comprobar si hay interrupciones activas
FINRTI	MOVE.L (A7)+,D1	*Recuperamos el valor de los registros que habiamos guardado en pila
	MOVE.L (A7)+,D0	
	RTE	*Fin rutina de tratamiento de excepcion

3. Casos de prueba

Para la correcta implementación de las subrutinas, diseñamos los siguientes casos de prueba, con los cuales pudimos avanzar paso a paso hasta llegar a ejecutar el código en el programa principal del caso de prueba de la RTI, con el cual, una vez completada la implementación de las demás subrutinas, hicimos el grueso de las pruebas utilizando puntos de ruptura para saber qué nos devolvía las subrutinas en cada momento.

3.1. Casos de prueba para INIT

En el caso de prueba de INIT se modifican las cadenas de bits de las direcciones de memoria a configurar, luego se hace una llamada a INIT y se comprueba si todo se ha inicializado correctamente. Es importante recalcar que para superar todas las pruebas del INIT la dirección de la RTI debe estar en la tabla de vectores para que al corromper la dir \$100 y luego llamar a la subrutina INIT supere todas las pruebas con éxito.

3.2. Casos de prueba para SCAN

En el primer caso de prueba de SCAN definimos un descriptor incorrecto (mayor que 1). Para ello declaramos la variable DESC con un valor 2 y la pasamos a la subrutina de la siguiente forma:

```
TAMANO EQU 30
DESC EQU 2

INICIO: BSR     INIT           * Inicia el controlador
OTRO:   MOVE.W  #TAMANO, -(A7) * Prepara el tamaño
        MOVE.L  #$5000, -(A7) * Prepara la dirección del buffer
        MOVE.L  #DESC, -(A7)  * Prepara el descriptor
        BSR     SCAN          * Recibe la línea
        ADD.L   #8, A7        * Restaura la pila
        BRA     OTRO

        BREAK
```

Resultado:

D0 = FFFFFFFF

Caso en el que pasamos un descriptor incorrecto (menor que 0):

```
TAMANO EQU 30
DESC EQU -1

INICIO: BSR     INIT           * Inicia el controlador
OTRO:   MOVE.W  #TAMANO, -(A7) * Prepara el tamaño
        MOVE.L  #$5000, -(A7) * Prepara la dirección del buffer
```

```

MOVE.L #DESC, -(A7)      * Prepara el descriptor
BSR     SCAN              * Recibe la linea
ADD.L   #8, A7            * Restaura la pila
BRA     OTRO

BREAK

```

Resultado:

D0 = FFFFFFFF

Caso en el que le pasamos un tamaño incorrecto (menor que 0):

```

TAMANO EQU -1
DESC EQU 0

INICIO: BSR     INIT              * Inicia el controlador
OTRO:   MOVE.W  #TAMANO, -(A7)    * Prepara el tamaño
        MOVE.L  #$5000, -(A7)    * Prepara la dirección del buffer
        MOVE.L  #DESC, -(A7)     * Prepara el descriptor
        BSR     SCAN              * Recibe la linea
        ADD.L   #8, A7            * Restaura la pila
        BRA     OTRO

BREAK

```

Resultado:

D0 = FFFFFFFF

Caso en el que el tamaño es 0:

```

TAMANO EQU 0
DESC EQU 0

INICIO: BSR     INIT              * Inicia el controlador
OTRO:   MOVE.W  #TAMANO, -(A7)    * Prepara el tamaño
        MOVE.L  #$5000, -(A7)    * Prepara la dirección del buffer
        MOVE.L  #DESC, -(A7)     * Prepara el descriptor
        BSR     SCAN              * Recibe la linea
        ADD.L   #8, A7            * Restaura la pila
        BRA     OTRO

```

```
BREAK
```

Resultado:

D0 = 0

Caso en el que le pasamos un número de caracteres por la línea menor al tamaño especificado:

```
TAMANO EQU 30
DESC EQU 0

INICIO: BSR     INIT                * Inicia el controlador
OTRO:  MOVE.W  #TAMANO, -(A7)        * Prepara el tamaño
      MOVE.L   #$5000, -(A7)        * Prepara la dirección del buffer
      MOVE.L   #DESC, -(A7)         * Prepara el descriptor
      BSR     SCAN                  * Recibe la línea
      ADD.L   #8, A7                 * Restaura la pila
      BRA     OTRO

      BREAK
```

Caracteres transmitidos por la línea A: 0x30, 0x31, 0x32, 0x33, 0x34

Resultado:

Direcciones de memoria:

5000: 0x30, 0x31, 0x32, 0x33

5004: 0x34, 0x00, ...

D0 = 5

Caso en el que llenamos el buffer, es decir, pasamos un número de caracteres por la línea igual o mayor al tamaño especificado:

```
TAMANO EQU 30
DESC EQU 0

INICIO: BSR     INIT                * Inicia el controlador
OTRO:  MOVE.W  #TAMANO, -(A7)        * Prepara el tamaño
      MOVE.L   #$5000, -(A7)        * Prepara la dirección del buffer
      MOVE.L   #DESC, -(A7)         * Prepara el descriptor
      BSR     SCAN                  * Recibe la línea
      ADD.L   #8, A7                 * Restaura la pila
      BRA     OTRO
```

BREAK

Caracteres transmitidos por la línea A: 0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39 en 3 ocasiones.

Resultado:

Direcciones de memoria:

5000: 0x30, 0x31, 0x32, 0x33

5004: 0x34, 0x35, 0x36, 0x37

5008: 0x38, 0x39, 0x30, 0x31

...

501C: 0x38, 0x39, 0x30, 0x00

D0 = 1e

3.3. Casos de prueba para PRINT

A continuación, para diseñar los casos de prueba para PRINT, seguimos escribiendo en el mismo programa principal que hemos usado para los casos de prueba de SCAN.

Caso en el que le pasamos un descriptor incorrecto (mayor que 1):

```
TAMANO EQU 30
DESA EQU 0
DESB EQU 2

INICIO: BSR    INIT                * Inicia el controlador
OTRO:   MOVE.W #TAMANO, -(A7)      * Prepara el tamaño
        MOVE.L #$5000, -(A7)      * Prepara la direccion del buffer
        MOVE.L #DESA, -(A7)       * Prepara el descriptor
        BSR    SCAN                * Recibe la linea
        ADD.L  #8, A7              * Restaura la pila
        MOVE.W #TAMANO, -(A7)      * Prepara el tamaño
        MOVE.L #$5000, -(A7)      * Prepara la direccion del buffer
        MOVE.L #DESB, -(A7)       * Prepara el descriptor
        BSR    PRINT               * Imprime la linea
        ADD.L  #8, A7              * Restaura la pila
        BRA    OTRO

BREAK
```

Resultado:

D0 = FFFFFFFF

Caso en el que le pasamos un descriptor incorrecto (menor que 0):

```
TAMANO EQU 30
DESA EQU 0
DESB EQU -1

INICIO: BSR    INIT                * Inicia el controlador
OTRO:  MOVE.W  #TAMANO, -(A7)      * Prepara el tamaño
      MOVE.L  #$5000, -(A7)      * Prepara la direccion del buffer
      MOVE.L  #DESA, -(A7)       * Prepara el descriptor
      BSR     SCAN                * Recibe la linea
      ADD.L   #8, A7              * Restaura la pila
      MOVE.W  #TAMANO, -(A7)      * Prepara el tamaño
      MOVE.L  #$5000, -(A7)      * Prepara la direccion del buffer
      MOVE.L  #DESB, -(A7)       * Prepara el descriptor
      BSR     PRINT               * Imprime la linea
      ADD.L   #8, A7              * Restaura la pila
      BRA     OTRO

      BREAK
```

Resultados:

D0 = FFFFFFFF

Caso en el que le pasamos un tamaño incorrecto (menor que 0):

```
TAMANO EQU -1
DESA EQU 0
DESB EQU 1

INICIO: BSR    INIT                * Inicia el controlador
OTRO:  MOVE.W  #TAMANO, -(A7)      * Prepara el tamaño
      MOVE.L  #$5000, -(A7)      * Prepara la direccion del buffer
      MOVE.L  #DESA, -(A7)       * Prepara el descriptor
      BSR     SCAN                * Recibe la linea
      ADD.L   #8, A7              * Restaura la pila
      MOVE.W  #TAMANO, -(A7)      * Prepara el tamaño
      MOVE.L  #$5000, -(A7)      * Prepara la direccion del buffer
      MOVE.L  #DESB, -(A7)       * Prepara el descriptor
```

```

BSR    PRINT          * Imprime la linea
ADD.L  #8,A7          * Restaura la pila
BRA    OTRO

BREAK

```

Resultados:

D0 = FFFFFFFF

Caso en el que le pasamos un tamaño igual a 0:

```

TAMANO EQU 0
DESA EQU 0
DESB EQU 1

INICIO: BSR    INIT          * Inicia el controlador
OTRO:   MOVE.W  #TAMANO,-(A7) * Prepara el tamaño
        MOVE.L  #$5000,-(A7) * Prepara la direccion del buffer
        MOVE.L  #DESA,-(A7)  * Prepara el descriptor
        BSR     SCAN         * Recibe la linea
        ADD.L   #8,A7        * Restaura la pila
        MOVE.W  #TAMANO,-(A7) * Prepara el tamaño
        MOVE.L  #$5000,-(A7) * Prepara la direccion del buffer
        MOVE.L  #DESB,-(A7)  * Prepara el descriptor
        BSR     PRINT        * Imprime la linea
        ADD.L   #8,A7        * Restaura la pila
        BRA     OTRO

BREAK

```

Resultados:

D0 = 0

3.4. Casos de prueba para RTI

Para llevar a cabo las pruebas de la subrutina RTI ha hecho falta diseñar un programa principal que permita ver en todo momento el tratamiento de las interrupciones y el comportamiento del programa en las subrutinas SCAN y PRINT.

A continuación está el programa principal utilizado para llevar a cabo todas las pruebas con las que hemos conseguido finalizar el proyecto superando todos los casos propuestos por el corrector del gestor de prácticas.

```
BUFFER: DS.B 2100 * Buffer para lectura y escritura de caracteres
PARDIR: DC.L 0 * Direccion que se pasa como parametro
PARTAM: DC.W 0 * Tamano que se pasa como parametro
CONTC: DC.W 0 * Contador de caracteres a imprimir
DESA: EQU 0 * Descriptor linea A
DESB: EQU 1 * Descriptor linea B
TAMBS: EQU 30 * Tamano de bloque para SCAN
TAMBP: EQU 7 * Tamano de bloque para PRINT

INICIO: BSR INIT

        MOVE.W #$2000,SR * Permite interrupciones
BUCPR:  MOVE.W #TAMBS,PARTAM * Inicializa parametro de tamano
        MOVE.L #BUFFER,PARDIR * Parametro BUFFER = comienzo del buffer
OTRAL:  MOVE.W PARTAM,-(A7) * Tamano de bloque
        MOVE.W #DESA,-(A7) * Puerto A
        MOVE.L PARDIR,-(A7) * Direccion de lectura
ESPL:   BSR SCAN

        ADD.L #8,A7 * Restablece la pila
        ADD.L D0,PARDIR * Calcula la nueva direccion de lectura
        SUB.W D0,PARTAM * Actualiza el numero de caracteres leidos
        BNE OTRAL * Si no se han leido todas los caracteres del bloque se vuelve a leer
        MOVE.W #TAMBS,CONTC * Inicializa contador de caracteres a imprimir
        MOVE.L #BUFFER,PARDIR * Parametro BUFFER = comienzo del buffer
OTRAE:  MOVE.W #TAMBP,PARTAM * Tamano de escritura = Tamano de bloque
ESPE:   MOVE.W PARTAM,-(A7) * Tamano de escritura
        MOVE.W #DESB,-(A7) * Puerto B
        MOVE.L PARDIR,-(A7) * Direccion de escritura
        BSR PRINT
        ADD.L #8,A7 * Restablece la pila
        ADD.L D0,PARDIR * Calcula la nueva direccion del buffer
        SUB.W D0,CONTC * Actualiza el contador de caracteres
        BEQ SALIR * Si no quedan caracteres se acaba
        SUB.W D0,PARTAM * Actualiza el tamano de escritura
        BNE ESPE * Si no se ha escrito todo el bloque se insiste
        CMP.W #TAMBP,CONTC * Si el no de caracteres que quedan es menor que el tamano
                               establecido se imprime ese numero
```

```
BHI OTRAE * Siguiente bloque  
MOVE.W CONTC,PARTAM  
BRA ESPE * Siguiente bloque  
SALIR: BRA BUCPR
```

Es importante mencionar el uso del fichero traza.log para detectar los errores que provocaban las últimas pruebas, especialmente a la hora de pasar por la línea cadenas de caracteres de gran longitud, gracias a la lectura de este fichero pudimos implementar la subrutina con éxito y superar todas las pruebas.

4. Conclusiones

Como conclusión final, creemos que este proyecto quizás tiene un difícil comienzo, ya que hemos pasado muchas horas leyendo y comprendiendo el manual para poder saber cómo debíamos desarrollar cada parte. Sin embargo, resulta muy gratificante implementar cada subrutina entendiendo que hace y conseguir que estas finalmente funcionen.

En cuanto al tiempo de realización, no sabríamos decir exactamente. El tiempo de programación como tal no ha sido elevado, como hemos mencionado antes, lo que creemos que ha sido el grueso del tiempo de realización del proyecto es el estudio y anotaciones previas del manual para poder entender y programar las subrutinas. Además de muchas horas depurando código para encontrar errores, que en su mayoría, eran fruto de despistes al programar y no de la lógica que seguía el programa. Tras todo esto, estimamos que la realización del proyecto nos ha tomado en torno a 60 horas entre los dos. Quizás pueda parecer bastante tiempo, pero como hemos mencionado antes, algunos errores muy simples nos han complicado bastante que las subrutinas funcionasen correctamente.

Pese al esfuerzo que supone este proyecto, creemos que nos ha servido para comprender mejor el funcionamiento de las operaciones de entrada y salida de periféricos por interrupciones, además de mejorar nuestras habilidades de depuración de código.